

# Reflections on Surrogate-Assisted Search-Based Testing: A Taxonomy and Two Replication Studies based on Industrial ADAS and Simulink Models

Shiva Nejati <sup>a,\*</sup>, Lev Sorokin <sup>b</sup>, Damir Safin <sup>b</sup>, Federico Formica <sup>c</sup>, Mohammad Mahdi Mahboob <sup>c</sup>, Claudio Menghi <sup>c,d</sup>

<sup>a</sup> University of Ottawa, Ottawa, K1N6N5, Canada

<sup>b</sup> Fortiss, Munich, Germany

<sup>c</sup> McMaster University, 1280 Main St W, Hamilton, ON L8S4L8, Canada

<sup>d</sup> University of Bergamo, via Salvecchio 19, Bergamo, 24129, Italy

## ARTICLE INFO

### Keywords:

Search-based testing  
Surrogate models  
Advanced driver assistance systems  
Simulink models  
Simulators  
Replications  
Evaluation metrics

## ABSTRACT

**Context:** Surrogate-assisted search-based testing (SA-SBT) aims to reduce the computational time for testing compute-intensive systems. Surrogates enhance testing techniques by improving test case generation focusing the testing budget on the most critical portions of the input domain. In addition, they can serve as approximations of the system under test (SUT) to predict test results instead of executing the tests on compute-intensive SUTs.

**Objective:** This article reflects on the existing SA-SBT techniques, particularly those applied to system-level testing and often facilitated using simulators or complex test beds. Recognizing the diversity of heuristic algorithms and evaluation methods employed in existing SA-SBT techniques, our objective is to synthesize these differences and present a comprehensive view of SA-SBT solutions. In addition, by critically reviewing our previous work on SA-SBT, we aim to identify the limitations in our proposed algorithms and evaluation methods and to propose potential improvements.

**Method:** We present a taxonomy that categorizes and contrasts existing SA-SBT solutions and highlights key research gaps. To identify the evaluation challenges, we conduct two replication studies of our past SA-SBT solutions: One study uses industrial advanced driver assistance system (ADAS) and the other relies on a Simulink model benchmark. We compare our results with those of the original studies and identify the difficulties in evaluating SA-SBT techniques, including the impact of different contextual factors on results generalization and the validity of our evaluation metrics.

**Results:** Based on our taxonomy and replication studies, we propose future research directions, including re-considerations in the current evaluation metrics used for SA-SBT solutions, utilizing surrogates for fault localization and repair in addition to testing, and creating frameworks for large-scale experiments by applying SA-SBT to multiple SUTs and simulators.

## 1. Introduction

To test complex and emerging systems at scale, the use of simulators is necessary. Simulators provide a flexible, effective, and efficient infrastructure to ensure the safety of complex systems compared to testing deployed systems in their operational environment (e.g., vehicles' on-road testing). However, executing each simulation scenario still takes a non-negligible amount of time, particularly when simulators need to be real-time such as in the case of simulators for autonomous vehicles. Since simulators' input spaces are very large

and high-dimensional, we require techniques to make simulation-based testing scalable. For example, non-trivial simulations of an industrial model of a satellite system, capturing the satellite behavior for 24 h, take, on average, around 84 min (~1.5 h) [1]. To test this satellite model for all of its parameters and inputs, we need to execute hundreds of such simulations, which can take months or even years to complete.

Search-based testing (SBT) has been traditionally used as an effective and efficient guidance for test generation performed at system-level [2]. SBT algorithms provide flexibility in representing the input

\* Corresponding author.

E-mail addresses: [snejati@uottawa.ca](mailto:snejati@uottawa.ca) (S. Nejati), [sorokin@fortiss.org](mailto:sorokin@fortiss.org) (L. Sorokin), [safin@fortiss.org](mailto:safin@fortiss.org) (D. Safin), [formicaf@mcmaster.ca](mailto:formicaf@mcmaster.ca) (F. Formica), [mahbom2@mcmaster.ca](mailto:mahbom2@mcmaster.ca) (M.M. Mahboob), [claudio.menghi@unibg.it](mailto:claudio.menghi@unibg.it) (C. Menghi).

<https://doi.org/10.1016/j.infsof.2023.107286>

Received 25 September 2022; Received in revised form 13 May 2023; Accepted 16 June 2023

Available online 24 June 2023

0950-5849/© 2023 Elsevier B.V. All rights reserved.

search space and choosing a strategy to traverse a subset of the search space. To scale SBT to handle compute-intensive cyber-physical systems (CPS) with large and multi-dimensional input spaces, a promising approach is to combine SBT with *surrogate models* [3] that enable effectively guiding the search exploration.

Surrogate models can enhance SBT in two ways: (1) Surrogate models identify the most critical regions of the input domain (i.e., the regions that include tests revealing critical faults) and sample test inputs that are in these regions. (2) Surrogate models predict simulation results instead of computing them. In both cases, surrogate models are generated and iteratively refined based on the intermediary test executions. As surrogate models improve through successive refinements, their predictions can help improve the search guidance. Alternatively, their predictions are used in lieu of actual test executions to reduce the search computation time by executing fewer tests.

This article reflects on four papers published between 2014 and 2021 [1,4–6] that propose surrogate-assisted search-based testing (SA-SBT) techniques for autonomous systems. The papers evaluated their proposed testing techniques on industrial Advanced Driving Assistance Systems (ADAS) [5,6], a satellite system, and an open-source benchmark of Simulink models [1] as well as an industrial controller from the automotive domain [4]. Since these papers are the main subjects of our analysis, we refer to these four papers as the *subject papers* hereafter in this article.

The subject papers apply SBT enhanced using surrogate models to system-level testing and demonstrate that this combination is effective for complex, emerging case studies such as self-driving and satellite systems. These subject papers complement existing SBT techniques that focus on testing software code and unit-level testing, and the research on testing and verification of CPS based on formal methods [7]. These four papers, while studying CPS with different behaviors and varying degrees of autonomy and complexity, motivate the need for SA-SBT techniques as follows: In the context of CPS and given the safety-critical nature of these systems, system-level testing is typically performed using simulators. CPS simulators, particularly those that are real-time, are compute-intensive, and their input spaces are large and high-dimensional. SA-SBT techniques are then proposed to scale simulation-based testing for CPS.

The four subject papers adopt an empirical approach and evaluate their SA-SBT techniques using case studies from different domains. The techniques that these papers propose, although all can be categorized as SA-SBT, vary in several details. For example, three of the subject papers [1,4,5] use surrogates to reduce the computation time of computing fitness functions, while the fourth paper [6] uses surrogates to guide the search exploration more effectively. Given the differences in case studies and proposed techniques, it is uncertain how the results can be generalized to similar systems. In this reflection paper, we review our previous research and attempt to answer two questions:

**RQ1** What are the different types and variations of SA-SBT techniques?

**RQ2** To what extent can we reproduce the results of these four subject papers?

To answer the first question, we propose a taxonomy for SA-SBT techniques and use it to categorize the subject papers and some recent papers that build on our research. To answer the second question, we report two replication studies that attempt to reproduce the results of two of the subject papers [1,6]. In particular, one replication study applies the SA-SBT solution proposed by Abdesslem et al. [6] to an industrial ADAS. For the second replication study, we reflect on the results and our experience in replicating the results generated by ARISTEO [1], an approximation-refinement testing technique based on surrogate models, in three consecutive editions (2020 [8], 2021 [9], and 2022 [10]) of the ARCH competition [11]. The ARCH competition is a friendly yearly competition between testing tools for continuous and hybrid systems [11].

We use Kolb's model of experiential learning [12] as a framework for our reflection study, which includes a taxonomy and two replication experiments. The Kolb model, shown in Fig. 1, outlines the steps of experiential learning and emphasizes the role of reflection in turning direct experiences into abstract concepts and knowledge that drive further research. The process starts with obtaining concrete experience, followed by reflective observations in light of existing knowledge. Reflection should focus on any discrepancies between existing knowledge and the new experience. From reflections, the researcher develops observations into abstract concepts and generalizations, and tests these through active experimentation.

We structure our reflection study following the experiential learning loop in Fig. 1. We develop a taxonomy (Section 2) that provides a set of abstract concepts and categories to help researchers classify and compare different SA-SBT research. We use the taxonomy to reflect on the current state of research and practice in the SA-SBT field, and to identify current research gaps (Section 3). We replicate two of our subject papers [1,6] and present our observations based on the replication studies (Section 4). Our findings, which include the taxonomy, the categorization of the state-of-art along with the taxonomy dimensions, and two replication studies lead us to identify a number of shortcomings and challenges that impact the research on SA-SBT. We discuss future research directions that the software testing researchers can embark on to improve and unify the research in this field. Section 5 presents our recommendations for future research which are listed as follows: (1) standardizing metrics for evaluating SA-SBT solutions, (2) using surrogates to improve fault localization and repair, (3) creating frameworks for large-scale and consistent application of SA-SBT across multiple SUTs and simulators, and (4) developing benchmarks and organizing international competitions to promote the continuous replication of research tools.

## 2. Taxonomy

This section presents our taxonomy for SA-SBT research and classifies the existing research along these categories. Fig. 2 presents a mind map diagram representing our taxonomy. The diagram visualizes the concepts captured by our taxonomy and their relations. SA-SBT (the center node of the mind map diagram) is connected to eight categories representing relevant problems to be considered in the design of SA-SBT techniques (purpose, usage, subject, design, type, context, scope, and evaluation metrics). Each problem is connected with sub-categories nodes representing relevant solutions and parameters to be considered when addressing these problems. In the following, we first describe a brief background on search-based testing (SBT). We then describe the categories of our taxonomy and discuss the sub-categories related to each of these categories.

**Background.** SBT relies on metaheuristic search [13] to generate a limited and effective set of test cases. Briefly, SBT algorithms work by building a set of initial candidate tests, iteratively applying tweak operations to one or more candidates to create new ones, and using a fitness function to decide whether to keep a candidate test for future iterations or to discard it.

**Purpose.** It refers to the way surrogates are combined with the metaheuristic search algorithm underlying an SBT technique and the search elements that they aim to improve. We identify three different purposes for surrogates: (1) *fitness computation*, (2) *test generation guidance*, and (3) *individuals selection*.

- *Fitness computation.* Surrogates reduce the fitness computation time by approximating the procedure for computing fitness values that often requires simulating the SUT.
- *Test generation guidance.* Surrogates replace or augment search operators (a.k.a. tweak operators). Specifically, they help identify the regions of the search space that include fitter individuals. New candidates can thus be generated from within these regions.

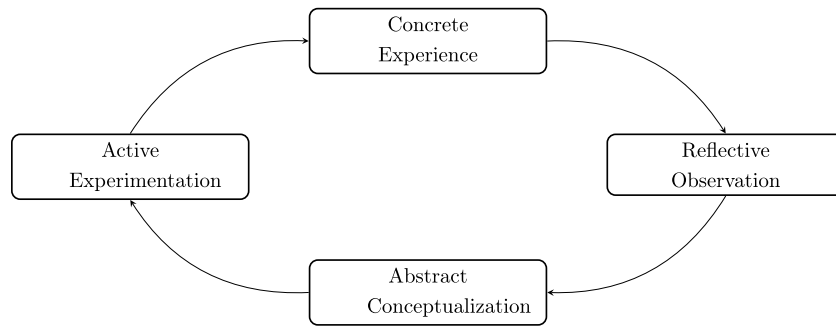


Fig. 1. Experiential learning cycle.



Fig. 2. A Taxonomy for Surrogate-Assisted Search-Based Testing (SA-SBT).

- **Individuals selection.** Decisions during the search process can be replaced with decisions guided by surrogates. For example, surrogates can be used to select fitter individuals for the initial population at the start.

**Usage.** It refers to the way surrogates improve the test generation approach of an SBT technique. We can use *individual* surrogates or an *ensemble* of surrogates. Surrogates may completely replace the SUT for test generation, or they may filter poor candidate test cases, while the

more promising candidates are evaluated on SUT. We identify three different purposes for surrogates: (1) *test case generation using single surrogates*, (2) *test case filtering using single surrogates*, and (3) *test case generation using ensemble surrogates*.

- *Test case generation using single surrogates*. Surrogates are used to generate candidate test cases without any need to execute the original SUT during the test generation. To determine if the generated tests should be kept or discarded, they are assessed on the original SUT only at the end.
- *Test case filtering using single surrogates*. Surrogates are used to prune candidates that have no chance of surviving the search selection process. They only discard poor test cases. The remaining test cases are assessed on the original SUT.
- *Test case generation using ensemble surrogates*. An ensemble of surrogates is used to assess individual test cases. A voting mechanism is used to determine, for a given test case, if the surrogates are precise enough, i.e., the surrogates produce the same (or close) fitness values. Otherwise, if the surrogates produce drastically different fitness values, the test case is re-evaluated on the SUT.

**Subject.** It refers to the artifacts approximated by a surrogate model. We identify two kinds of artifacts to which surrogates can be applied: *model* and *model&requirement assessment approximation*.

- *Model*. The model approximation technique relies on the creation of an approximation of the SUT that closely mimics its behavior, but is significantly cheaper to execute.
- *Model&requirement-assessment*. It relies on a function to predict whether test cases satisfy or violate the system requirements. Surrogates may approximate both the dynamics of the SUT and the assessment of the requirement or they may only capture the assessment of the requirement.

**Design.** It refers to the process used to build a surrogate model. We identify two processes for the design of surrogate models: *manual* and *automatic*.

- *Manual*. In the manual process, engineers define a surrogate model by relying on their own experience and knowledge of SUT.
- *Automatic*. In the automatic process, engineers rely on some measurements to automatically construct a surrogate model. For example, these measurements can include logs recording the inputs and outputs of the SUT. We consider two variations for the automatic process: *dynamic* and *static*. The dynamic process improves surrogate models over time (e.g., thorough an approximation-refinement loop). The static process does not improve or modify surrogate models after they are computed.

**Type.** It captures the characteristics of surrogate models, and has two dimensions: the *Model type* and the *Output type*.

- *Model Type*. It captures the degree of information we have about the structure of surrogate models which can be one of the following three options: *white box*, *gray box*, and *black box*. The white box type indicates that surrogates are designed manually based on the knowledge that engineers have about the SUT. Hence, there is full observability to their structure. Black box models are used when the structure of the surrogate model is unknown, and engineers do not have any knowledge about the SUT, such as the physics and control laws that regulate the system behavior. Gray box models are used when there is partial information about the structure of surrogate models. For example, gray box models are useful when the engineer knows the physics and control laws that regulate the system behavior but does not know some parameters, e.g., the heat transfer coefficient of a car engine. Automatic techniques are needed to compute black box models and unknown parameters of gray box models.

- *Output Type*. It refers to the format of the outputs of surrogate models. We recognize two types of outputs: *Single Data* and *Time Series*. Some surrogate models produce a single data point as output, e.g., a Boolean value indicating whether a property is satisfied or a real value indicating a degree of satisfaction. Alternatively, surrogate models may generate time series data that is a sequence of data points indexed in time order.

**Context.** It refers to the context where surrogates are used. We identify three different usage contexts for surrogates: *Model-in-the-Loop (MiL)*, *Software-in-the-Loop (SiL)*, and *Hardware-in-the-Loop (HiL)*.

- *Model-in-the-Loop*. It concerns with surrogate models that approximate a virtual and high-level representation of the system, e.g., some type of software model.
- *Software-in-the-Loop*. It relates to surrogate models that approximate software code.
- *Hardware-in-the-Loop*. It is about surrogate models that are built based on the measurements obtained from the actual operating system including its hardware components.

**Scope.** It refers to the portion of the search space surrogate models approximate. The search domain a surrogate model targets can be *global* or *local*.

- *Global*. When surrogate models capture a global view of the search space of the SUT, they approximate the entire search space, and hence, enable an explorative search.
- *Local*. It refers to the case in which surrogate models capture a local view by approximating a region of the search space that includes desired information. In that case, surrogates are utilized for an exploitative search.

**Evaluation Metrics.** It refers to the metrics used to assess an SBT approach. We identify three metrics: *Number of Iterations*, *Execution Time*, and *Estimated Execution Time*.

- *Number of Iterations*. It is the number of times an SBT approach executes the SUT.
- *Execution Time*. It measures the time the approach requires to detect a requirement violation.
- *Estimated Execution Time*. It predicts the time the approach would take to be applied to a compute-intensive system based on the results obtained for non-compute-intensive systems.

We will reflect on these metrics in detail in Section 5.

Some elements of our taxonomy in Fig. 2 are specific to SA-SBT techniques, while some are relevant to SBT in general. In particular, the *context*, *scope* and *purpose* categories can be used to classify SBT techniques and are not particularly tied to SA-SBT methods. The other categories, however, have some elements or sub-categories that make them relevant to SA-SBT only.

### 3. Positioning against the state of the art and practice

To position the existing literature against the state of the arts and practice, one of the authors collected existing papers and classified relevant research literature along our taxonomy categories. We considered 16 papers, the four subject papers of our study [1,4–6] and a set of 12 papers by other researchers that employ SBT for testing complex software systems identified by forward snowballing [14–25], and classified relevant research literature along our taxonomy categories. We relied on Google Scholar for the forward snowballing: we identified new papers by consulting papers citing the four subject papers, we reviewed the retrieved papers, and we selected the papers that proposed a new SBT approach. The forward snowballing activity lead to 12 papers, including one paper [22] proposing fuzz testing techniques and six papers presenting falsification techniques [17–21].



**Table 1**  
Classifying the SA-SBT techniques based on our taxonomy.

| Category    | References                                                                                                          |
|-------------|---------------------------------------------------------------------------------------------------------------------|
| Purpose     | Fitness computation [4,5,14–16,19,21,23–25]<br>Test Generation Guidance [6,18,20]<br>Individual Selection [1,17,22] |
| Usage       | TC generation–single [1,6,14,15,17,19,22,24,25]<br>TC filtering [4,5,18]<br>TC generation–ensemble [16,20,21,23]    |
| Subject     | Model [1,14,17–19,22]<br>Model and requirement–assessment [4–6,15,16,20,21,23]                                      |
| Design      | Manual [15,20]<br>Automatic–Static<br>Automatic–Dynamic [1,4–6,14,16–19,21–23]                                      |
| Model Type  | White Box [1,14,17,18,20,21]<br>Gray Box [1,21]<br>Black Box [1,4–6,15,16,19,21–25]                                 |
| Output Type | Single Data [4–6,15,16,20–25]<br>Time Series [1,14,17–19]                                                           |
| Context     | MiL [1,4–6,14–25]<br>SiL<br>HiL [14]                                                                                |
| Scope       | Global Search [1,5,6,14–25]<br>Local Search [4,16,19,23]                                                            |
| Metric      | Estimated Execution Time [1]<br>Execution Time [4–6,14,15,20,21,23]<br>Num of Iterations [1,22]                     |

These works are specific instances of SBT techniques: fuzz testing is an SBT technique that injects unexpected inputs into a system to reveal software defects, and falsification techniques are SBT techniques driven by a requirement expressed using a formal language. We used these papers to demonstrate example research for different aspects of our taxonomy, and also, to analyze what sub-categories of our taxonomy are lesser studied compared to the others. Table 1 classifies relevant research literature along our taxonomy categories. Note that some papers are listed for some categories in Table 1. For example, for the evaluation metrics category, we do not list papers that do not evaluate their solutions. Table 1 shows that the SA-SBT techniques we reviewed cover all the nine top-level categories, and only two sub-categories (the Automatic-Static sub-category of the design category, and the SiL sub-category of the Context category) are not covered. In the following, we discuss the main findings for each category.

**Purpose.** Most of the techniques we reviewed use surrogates for fitness computation (10 out of 16), followed by test generation guidance (3 out of 16) and individual selection (3 out of 16). Although research literature has focused less on using surrogates for test generation guidance and individual selection compared to fitness computation, incorporating surrogates can greatly enhance SBT techniques by pinpointing critical regions of the test input domain and selecting the most crucial individuals (test cases).

**Usage.** Most of the techniques we reviewed use test case generation using single surrogates (9 out of 16), followed by test case generation using ensemble surrogates (4 out of 16), and test case filtering using single surrogates (3 out of 16). Although techniques that use single surrogates for test case generation are prevalent, surrogate models are also used for other testing activities (i.e., test case filtering) and aggregated into ensembles. Since surrogate models are a relatively new technology in SBT, their potential usages are still unclear and will grow over time.

**Subject.** For 6 out of 16 papers, the subject of the surrogate model is the original model under test, while for the remaining eight papers, it includes both the model and the requirement assessment. Using a surrogate model to approximate the original model under test and a fitness function to evaluate the different test cases ensures that the system's dynamics are approximated while the fitness computation is accurate.

Differently, also considering an approximation of the fitness function introduces additional noise. There is a need for more empirical studies to precisely identify the situations where one of these approaches is preferred.

**Design.** Most of the techniques we reviewed (12 out of 16) automatically compute the surrogate model and use dynamic processes that improve the surrogate model over time. Two techniques manually defined the surrogate model. None of the techniques we reviewed used a static surrogate model, i.e., a fixed surrogate model that is not improved over time. Although automatic techniques do not require manually defining the surrogate model, in many applications, the manual design offers several benefits since engineers can incorporate knowledge about the structure of the SUT in the design of the surrogates. More empirical studies are needed to identify the scenarios in which one technique must be preferred over the other.

**Model Type.** Most of the techniques we reviewed are black box (8 out of 16), followed by white box (4 out of 16). Two techniques, Menghi et al. [1] and Humeniuk et al. [21], are generic and rely on other technologies such as system identification (SI) [26,27] that enable them to support black box, white box, and gray box model types. In particular, SI enables us to create surrogates that belong to all the three categories. Although less studied, gray box technologies can offer a good trade-off in situations in which engineers have some knowledge about the system's internal structure and need to be aware of the values of some of the parameters that still need to be discovered.

**Output Type.** For 9 out of 16 techniques, the output type is Single Data. The remaining five techniques use time series. Therefore, in most cases, the surrogates directly predict the single data fitness value for a test input. Surrogates that use time series as output type are more recent [1]. Time series data leads to the generation of more data points that help with better training of surrogate models.

**Context.** For all the techniques we considered, the surrogate context is MiL. One technique [14] claims that their approach also supports HiL in addition to MiL. The prevalence of MiL techniques is justified by the fact that MiL relies on simulators and system models, which are often publicly available. In contrast, SiL and HiL involve running the software under test on real or emulated hardware and require direct access to the source code and system hardware, which can be challenging in an industrial context. To overcome this hurdle, close collaboration with industry partners is often necessary. Furthermore, creating the necessary hardware infrastructure and setup can be both costly and time-consuming, particularly in an academic setting.

**Scope.** The scope of 10 out of 16 techniques is global search. The scope of one techniques is exclusively local search. Three techniques consider both for local and global search. We noticed that the boundary between global and local search was often blurry. Additional research work is needed to precisely identify and define the boundary between the local and global search for SBT.

**Evaluation Metric.** Most of the techniques use execution time as an evaluation metric (8 out of 16), followed by the number of iterations (2 out of 14), and the estimated execution time (1 out of 14). Some techniques either did not use any evaluation metric, as they propose theoretical solutions which are not empirically evaluated, or their evaluation did not involve a comparison of SA-SBT solutions. Although the execution time measures the actual time required for fault-finding, it makes experiments hardly reproducible as it depends from the actual hardware platform on which the experiments are executed. On the contrary, the number of iterations enables experiment replication as the hardware platform does not influence it. However, the execution time of the iterations may differ across different techniques. We provide additional reflections and considerations on the evaluation metric category in Section 5.

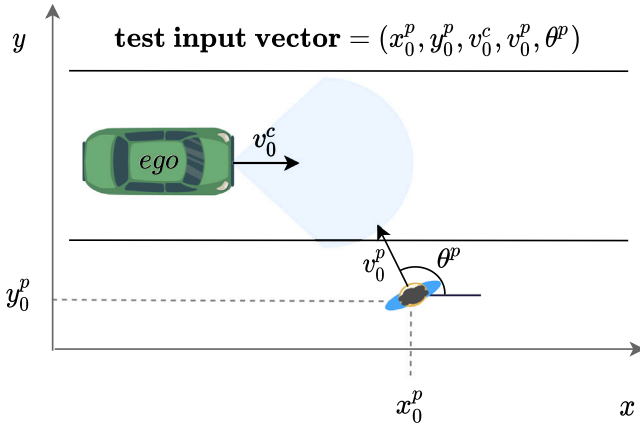


Fig. 3. SUT of the original study [6]: a pedestrian crosses the lane of the ego car equipped with the AEB system.

#### 4. Replication studies

Papers listed in Table 1 do not perform any replication of the prior studies. In the next two sections, we present two replication studies based on two of the subject papers. Specifically, we report replication studies of NSGAII-DT [6] and ARISTEO [1]. NSGAII-DT and ARISTEO belong to different sub-categories under the purpose, subject, model type, output type, and evaluation metric categories in our taxonomy. Replicating these complementary approaches allows us to gain insightful observations and lessons learned.

##### 4.1. Replication study: ADAS case study

In this section, we report on the replication of the experiments from the subject paper proposing the NSGAII-DT technique, which was evaluated using an industrial ADAS case study [6]. The replication attempt was performed by different researchers (the second and third authors of the paper) and by considering an ADAS which has been developed in a joint european project [28] in collaboration with different academic and industrial partners. The system is different from that used in the original paper [6]. In the following, we first describe the experiment setup of the original and the replication studies, then we present our results.

###### 4.1.1. Experiment setup

**System under test (SUT).** The system under test for the original subject paper is the Automated Emergency Breaking (AEB) system presented in Fig. 3. The AEB identifies pedestrians in front of a vehicle and avoids collision by applying the brake when necessary. The ego car (i.e., the self-driving car equipped with AEB) drives from the left-end on an urban street, and a pedestrian starts crossing the street from the middle of the street.

For the replication study, we consider an Automated Valet Parking (AVP) system presented in Fig. 4: A driver leaves the car at the entrance of a parking lot and AVP autonomously parks the car in an available parking spot. The ego car is equipped with an AEB to ensure that the car does not collide with other objects, e.g., pedestrians or cars, and a path follower to assure that the ego vehicle follows a pre-computed path for parking. A Prescan internal planar vehicle dynamics model is used. The planar vehicle dynamics is mathematically the simplest model to determine dynamics of vehicles and assumes that the vehicle moves parallel to the ground and has no pitch or roll motions. The AVP is tested using a scenario where an occluded pedestrian appears in the path of the ego car as the ego car approaches a free parking spot. An occluded pedestrian is a pedestrian who is partially or completely

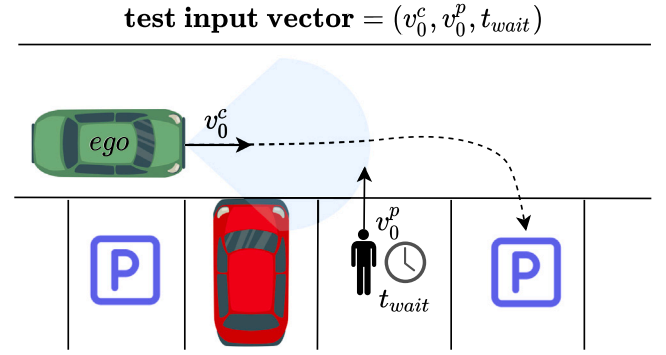


Fig. 4. AVP testing scenario from the replication study: a pedestrian occluded by a parking car is crossing the lane of the ego car, which is approaching a free parking spot.

obscured from the ego car's view by another object (e.g., a pillar or a parked vehicle).

**Search Space.** The original study tests are obtained by varying the following features of the AEB test scenario (see Fig. 3): The speed of the ego car ( $v_0^c$ ), the initial position of the pedestrian with respect to the ego car ( $x_0^p, y_0^p$ ), the speed ( $v_0^p$ ) and the orientation ( $\theta^p$ ) of the pedestrian, the weather type, which can be either normal, rainy and snowy, and the road shape, which can be either straight, curved or ramped. For the curved and ramped roads, the test input includes the curve radius and the ramp height. For the snowy and rainy weather types, the test input includes the level of precipitation and the presence of fog with different density levels.

The search space for the replication study varies the speed of the ego car ( $v_0^c$ ), the speed of the pedestrian ( $v_0^p$ ) and the time that the pedestrian starts running after the start of the simulation ( $t_{wait}$ ). The variable  $t_{wait}$  has a similar purpose as the variables ( $x_0^p, y_0^p$ ) in the original study. Both  $t_{wait}$  and ( $x_0^p, y_0^p$ ) help generate different tests where the ego car and the pedestrian appear in different proximities to one another. The AVP testing excludes road shape and weather-related factors, as they are not essential for assessing AVP functionality. Given the confined parking lot environment, fluctuations in these conditions are considered non-essential for evaluating system performance. In addition, the AVP testing scenario is focused on occluded pedestrians, and for them, the initial position and the orientation are not relevant since they are blocked from the ego car's field of view at the beginning. Instead, for occluded pedestrians, the relevant information for testing is when they appear in front of the ego car (variable  $t_{wait}$ ), which is included as a test input in the replication study.

Both the original and the replication studies use the PreScan simulator [29] to test their respective SUT. Both studies set 100 as the sampling rate for PreScan and generate simulations with the duration of 10s.

**Fitness functions.** Both studies define multiple fitness functions to assess the quality of individual simulations. Fitness functions measure how close the scenario is to a collision. The original study defines three fitness functions: ( $F_1$ ) The minimum distance between the pedestrian and the field of view of the ego car, ( $F_2$ ) The velocity of the ego vehicle at the time where the distance between the ego car and the pedestrian is at its lowest value, i.e., when the minimum for  $F_1$  is computed, and ( $F_3$ ) The certainty of the detection of a pedestrian. The three fitness functions collectively assess this requirement: "AEB accurately detects a pedestrian in front of the car, yet an accident occurs where the car strikes the pedestrian at a relatively high speed (i.e., over 30 km/h)". By minimizing  $F_1$  and maximizing  $F_2$ , we derive scenarios that represent either near-collision situations or high-speed collisions involving the ego car. The fitness function  $F_3$  ensures the prerequisite of our requirement: AEB detection of a pedestrian. If AEB fails to detect

**Table 2**  
Experimental setups for the replication study and the original study [6].

| Category              | Replication study setup                       | Original study setup                                    |
|-----------------------|-----------------------------------------------|---------------------------------------------------------|
| SUT                   | Planar vehicle dynamics, Path Follower, Lidar | Planar & roll vehicle dynamics, Object detection, Lidar |
| Sampling rate         | 1/100                                         | 1/100                                                   |
| Scenario              | Pedestrian crossing street                    | Pedestrian crossing street                              |
| # fitness functions   | 2                                             | 3                                                       |
| Initial sampling      | LHS                                           | Combinatorial Testing                                   |
| Mutation probability  | 1/n                                           | 1/n                                                     |
| Crossover probability | 0.6                                           | 0.6                                                     |

a pedestrian (i.e., low  $F_3$ ), the requirement is trivially satisfied. By maximizing  $F_3$ , we detect scenarios in which AEB detects pedestrians but cannot prevent a collision, thus violating the requirement.

The replication study defines two fitness functions: (1)  $F'_1$  (similar to  $F_1$ ) this function computes the minimum distance between the ego car and the pedestrian. However, in the replication study, this distance is defined to be the distance between the pedestrian and the front side of the car. Hence, the  $F'_1$  function is defined such that it assigns more optimal values to collisions where the pedestrian hits the car from the front. This is to provide more effective guidance for the search to generate scenarios where the collision is realistic and meaningful. That is, the front side of the ego car hits a pedestrian. (2)  $F'_2$  (similar to  $F_2$ ) this function captures the velocity of the ego vehicle at the time where the distance between the ego car and the pedestrian is at its lowest value. The replication study does not consider the third fitness function (i.e., the certainty of detection) as in the original study, because the AVP does not provide detailed output that measures the certainty of detection.

In addition to fitness functions, both studies define when a scenario represents a failure of the SUT. For both studies, a failure is when the ego car hits the pedestrian with a high speed. Failure revealing scenarios are identified by setting a threshold on  $F_1$  and  $F_2$  (resp.  $F'_1$  and  $F'_2$  in the replication study) to indicate that the car and the pedestrian collide and the car has a high speed at the time of the collision. Since the SUTs in the original and the replication studies are different, the threshold values for what defines a failure in these two studies are different too. Hence, the thresholds in these studies were defined independently and based on the requirements of their respective SUTs.

**Initial population.** The replication study uses Latin Hypercube Sampling [30], a quasi-random sampling strategy, to generate the initial population. The original study used combinatorial testing using the Pledge tool [31] to generate an initial population of diversified individuals. Given that the purpose of both strategies is to generate a diverse initial set of individuals, we believe the differences in generating the initial populations do not significantly impact the results of our study.

**Search Algorithm.** Both the original and the replication study rely on NSGAI-DT. NSGAI-DT uses machine learning decision tree models and multi-objective evolutionary search [13]. Decision tree models are developed based on the scenarios generated at intermediary search iterations. The models learn the characteristics of the critical test scenarios and identify critical regions in an input space (i.e., the regions of a test input space that are likely to contain more fault-revealing test cases and test cases that reveal more severe faults). The subsequent search iterations then focus on the critical regions, and select and evolve critical scenarios in those regions. When the search inside the critical regions is terminated, a new decision tree model is re-generated using the entire population. NSGAI-DT continues by iteratively building decision trees followed by search iterations focused on critical regions identified by the trees until the search time budget runs out. The original and replication studies consider the same values for the search parameters, such as mutation and crossover rates. The main parameters and setup elements of the replication and the original studies are listed in Table 2.

#### 4.1.2. Results

The original study provides two sets of metrics to evaluate the results of NSGAI-DT: (1) Quality indicators for Pareto-based search algorithms [32], and (2) the number of distinct critical scenarios representing failures of the SUT. As a Pareto-based search algorithm, NSGAI-DT aims to find a set of non-dominated solutions in a multi-objective optimization problem. These algorithms are typically assessed based on a special category of metrics, referred to as *quality indicators* [32] that compare Pareto-based search algorithm in terms of convergence, uniformity, spread and cardinality. In particular, the original study used three well-known quality indicators, Hypervolume (HV), Spread ( $\Delta$ ) and Generational Distance (GD), to compare the results of NSGAI-DT with those of NSGAI. Below we describe the experiment setup for our replication study and then compare and discuss the results of both studies.

**Setup.** We evaluate the performance of NSGAI-DT and NSGAI on the AVP case study for a minimum of 300 min. We note that NSGAI served as a baseline in the original study. The average execution time of a single AVP simulation is approximately 19 s, allowing for at least 1000 simulations (fitness evaluations) to be completed within the 300-min computation time. We rerun each of NSGAI-DT and NSGAI for 10 times to account for their randomness. The experiments were performed on a computer with an i7-8700 processor with 3.19 GHz (12 cores) and 32 GB RAM.

Note that the AEB simulations in the original study took significantly longer than the AVP simulations, with an average execution time of 1.2 min for AEB and 19 s for AVP. In the original study, both NSGAI-DT and NSGAI algorithms were run for a 24-h period, providing enough time for at least 1000 minimum AEB simulations, similar to our replication study.

**Metrics.** For NSGAI, we compute HV, Spread ( $\Delta$ ) and GD after each generation. But, for NSGAI-DT, we compute these metrics after each generation of each of the NSGAI algorithm runs performed inside the critical regions. HV measures the size of the space covered by the members of a Pareto front generated by a search algorithm [32]. The higher this size, the better the results of the algorithm. GD measures the Euclidean distance between members of the calculated Pareto front and the nearest solutions on a reference Pareto front [32]. The lower the value of GD, the more optimal the calculated Pareto front solutions are. Spread measures the extent of spread among the members of a Pareto front generated by a search algorithm [33]. The lower the Spread values, the better spread out the search outputs. In addition, we compute the number of distinct critical scenarios generated by NSGAI and NSGAI-DT, following the definition provided in the original study, where distinct scenarios are defined as those that differ in at least one input value [6].

**Data Availability.** The implementation of NSGAI and NSGAI-DT for AVP and the detailed results of these algorithms for our replication study are available online [34]. The source code of the AVP system can be not provided, as this is not an open-source system. There has been an agreement on referencing the AVP in this reflection study, as no implementation of the SUT is disclosed.

**Comparison of the results.** Figs. 5 and 6 show the results of the HV, Spread and GD metrics over time for NSGAI and NSGAI-DT for our original and replication study, respectively. The results for the original study are shown at every four-hour time interval starting at

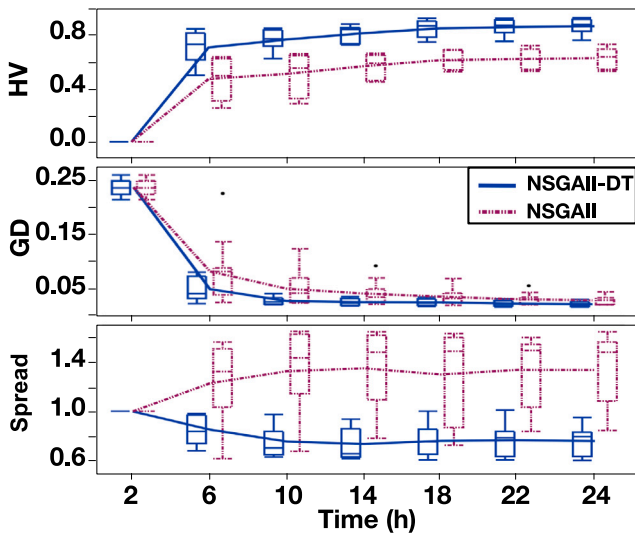


Fig. 5. The comparison of HV, GD and Spread ( $\Delta$ ) values obtained by NSGAI and NSGAI-DT from the original study [6].

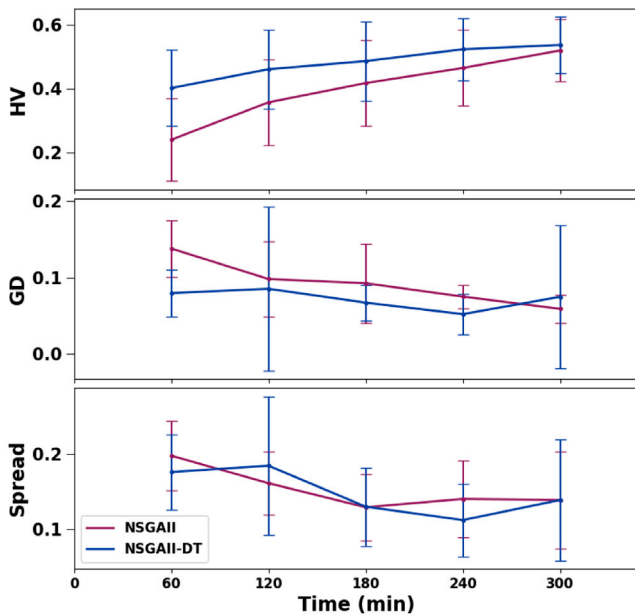


Fig. 6. The comparison of HV, GD and Spread values obtained by NSGAI and NSGAI-DT from the replication study.

2 h until the time limit of 24h, while the results for the replication study are shown at every 60 min starting from 60 min and until 300 min. Since the original and replication experiments are performed on different case studies with different inputs and fitness functions, we cannot make direct comparisons between the values of these metrics, but instead, we aim to compare the trends of these metrics over time.

In both the original and replication study, the HV values for NSGAI-DT are better than those of NSGAI at the start of the experiments. However, over time, the differences between the HV values of the two algorithms tend to decrease. In the replication study, this reduction is quite steep, with the HV values of NSGAI and NSGAI-DT eventually becoming equal. In contrast, in the original study, the HV values of the two algorithms seem to stabilize at a certain point. The GD values of NSGAI-DT are slightly better than those of NSGAI over the entire duration both in the original study and in the replication study. However, at the end of the simulation, which is 300 min, the GD values

Table 3

The number of distinct critical scenarios obtained by NSGAI and NSGAI-DT from the replication and original study.

|                      | Replication study | Original study |
|----------------------|-------------------|----------------|
| NSGAI                | 639               | 411            |
| NSGAI-DT             | 1733              | 731            |
| Ratio NSGAI-DT/NSGAI | 2.7               | 1.8            |

of NSGAI slightly improve over those of NSGAI-DT in the replication study. Further, the variations of the GD values for NSGAI-DT in the replication study are noticeably higher than those of the GD values of NSGAI-DT in the original study. There are remarkable differences between the replication and the original studies in the values of the Spread metric obtained for NSGAI-DT. While in the original study the values of Spread for NSGAI-DT are considerably better than those of NSGAI, for the replication study the values of Spread for both NSGAI and NSGAI-DT are almost the same.

Table 3 shows the number of distinct critical scenarios generated by NSGAI-DT and NSGAI in the replication and original studies. As shown in the table, in both studies, provided with the same search time budget, NSGAI-DT is able to find considerably more distinct critical scenarios compared to NSGAI. In particular, the number of distinct critical scenarios that NSGAI-DT finds are, respectively, 1.8 and 2.7 times more than those that NSGAI can find in the original and replication studies.

**Summary.** The replication study supports some of the conclusions of the original study. However, there are some disparities with some specific findings. Both studies confirm that the surrogate-based SBT method, NSGAI-DT, generates significantly more distinct failures than its non-surrogate counterpart within the same search time budget. However, the consistency of the results comparing the quality of the Pareto-based solutions generated by NSGAI and NSGAI-DT is questionable. The HV and GD results from the replication study partially align with those from the original study, whereas the results of the Spread metric are significantly different. We will discuss in Section 5.1 the reasons behind the inconsistency of the results for the quality indicators. Despite our efforts to align the two studies as much as possible, the SUTs, and hence, the test inputs and fitness functions remain to be different. These factors may have contributed to the discrepancies in the outcomes. Independently from the real sources of these differences, our replication study allowed us to gain a deeper understanding of the limitations and significance of the metrics used to evaluate NSGAI-DT, which we elaborate on in Section 5.

#### 4.2. Replication study: Simulink

In this section, we report on an attempt to replicate the experiments of ARISTEO, an approximation-refinement testing technique driven by surrogate models proposed by one of the subject papers [1]. The ARISTEO technique is showcased by a plugin of S-TaLiRo [35], a falsification-based testing tool for Simulink models. In their paper, the authors of ARISTEO, evaluated their contribution by considering a set of case studies which included some of the benchmark models of the ARCH competition [9] and one additional industrial case study from the satellite domain. The industrial case study is not publicly available since the authors could not release it due to a non-disclosure agreement. However, the authors provided a complete replication package with the other benchmark models and results. In addition, they participated in the ARCH competition, editions of 2020 [8], 2021 [9], and 2022 [10]. The ARCH competition is a friendly yearly competition between testing tools for continuous and hybrid systems [11]. The competition includes several participants, such as FaICAuN [36], S-TaLiRo [35], falsify [37], FalStar [38], ForeSee [39]. The models and requirements considered by the competition are publicly available and provided under different licenses.



**Table 4**  
Results for piecewise continuous input signals.

|        | 2020  |           |             | 2021  |           |             | 2022  |           |             |
|--------|-------|-----------|-------------|-------|-----------|-------------|-------|-----------|-------------|
|        | FR    | $\bar{S}$ | $\tilde{S}$ | FR    | $\bar{S}$ | $\tilde{S}$ | FR    | $\bar{S}$ | $\tilde{S}$ |
| AT1    | 0     | –         | –           | 0     | –         | –           | 0     | –         | –           |
| AT2    | 50    | 4.4       | 4           | 50    | 4.1       | 3           | 10    | 5.4       | 2.0         |
| AT51   | nSbyA |           |             | 0     | –         | –           | 0     | –         | –           |
| AT52   | nSbyA |           |             | 50    | 3.2       | 2           | 10    | 1.7       | 1.5         |
| AT53   | nSbyA |           |             | 50    | 2.6       | 2           | 10    | 12.8      | 8.5         |
| AT54   | nSbyA |           |             | 3     | 295.1     | 300         | 0     | –         | –           |
| AT6a   | 45    | 90.7      | 69          | 50    | 39.0      | 23          | 10    | 93.4      | 70.0        |
| AT6b   | 50    | 18.1      | 15          | 49    | 101.7     | 83          | 4     | 112.3     | 127         |
| AT6c   | 44    | 95.6      | 66          | 50    | 27.2      | 16          | 10    | 124.2     | 111.5       |
| AT6abc | NPofB |           |             | 50    | 27.2      | 16          | 10    | 73.4      | 30.5        |
| NN     | 50    | 62.8      | 46          | 50    | 62.8      | 46          | 1     | 299.0     | 299.0       |
| NNx    | NPofB |           |             | NPofB |           |             | 0     | –         | –           |
| WT1    | 50    | 15.6      | 10          | 50    | 13.0      | 10          | NPofB |           |             |
| WT2    | 50    | 1.5       | 1           | 50    | 1.3       | 1           | NPofB |           |             |
| WT3    | 50    | 3.4       | 3           | 50    | 2.4       | 2           | NPofB |           |             |
| WT4    | 50    | 1.0       | 1           | 50    | 1.0       | 1           | NPofB |           |             |
| CC1    | 50    | 16.1      | 11          | 50    | 27.0      | 15          | 10    | 24.8      | 17.5        |
| CC2    | 50    | 1.0       | 1           | 50    | 9.2       | 6           | 10    | 14.6      | 9.0         |
| CC3    | 50    | 45.8      | 27          | 50    | 56.3      | 46          | 10    | 29.5      | 35.0        |
| CC4    | 50    | 1.0       | 1           | 0     | –         | –           | 0     | –         | –           |
| CC5    | 49    | 52.5      | 40          | 50    | 25.9      | 17          | 10    | 18.8      | 18.0        |
| CCx    | NPofB |           |             | 15    | 250.0     | 300         | 4     | 134.0     | 74.0        |
| F16    | nSbyA |           |             | 0     | –         | –           | 0     | –         | –           |
| SC     | 50    | 1.0       | 1           | 0     | –         | –           | 0     | –         | –           |

**Table 5**  
Results for constrained input signals.

|        | 2020  |           |             | 2021 |           |             | 2022  |           |             |
|--------|-------|-----------|-------------|------|-----------|-------------|-------|-----------|-------------|
|        | FR    | $\bar{S}$ | $\tilde{S}$ | FR   | $\bar{S}$ | $\tilde{S}$ | FR    | $\bar{S}$ | $\tilde{S}$ |
| AT1    | 0     | –         | –           | 0    | –         | –           | 0     | –         | –           |
| AT2    | 50    | 4.5       | 3           | 50   | 5.1       | 4           | 10    | 12.9      | 8.0         |
| AT51   | nSbyA |           |             | 50   | 8.4       | 6           | 10    | 19.0      | 10.0        |
| AT52   | nSbyA |           |             | 50   | 3.5       | 3           | 9     | 74.7      | 46.0        |
| AT53   | nSbyA |           |             | 50   | 2.6       | 2           | 10    | 1.4       | 1.0         |
| AT54   | nSbyA |           |             | 50   | 29.4      | 17          | 10    | 44.0      | 40.0        |
| AT6a   | 41    | 116.3     | 72          | 47   | 103.1     | 80          | 7     | 65.1      | 62.0        |
| AT6b   | 50    | 36.3      | 27          | 45   | 164.7     | 174         | 5     | 91.2      | 74.0        |
| AT6c   | 44    | 89.8.6    | 73          | 49   | 89.1      | 60          | 7     | 175       | 141.0       |
| AT6abc | NPofB |           |             | 50   | 79.1      | 70          | 9     | 84.1      | 81.0        |
| AFC27  | 50    | 2.3       | 1           | 50   | 2.3       | 1           | 9     | 21.9      | 25.0        |
| AFC29  | 50    | 28.5      | 23          | 50   | 1.0       | 1           | 10    | 3.4       | 3.5         |
| AFC33  | 50    | 24.7      | 16          | 50   | 1.0       | 1           | 0     | –         | –           |
| NN     | 50    | 62.8      | 46          | 50   | 62.8      | 46          | 10    | 117       | 82.5        |
| NNx    | NPofB |           |             | 50   | 1.0       | 1           | 0     | –         | –           |
| WT1    | 50    | 1.4       | 1           | 50   | 1.4       | 1           | NPofB |           |             |
| WT2    | 50    | 1.0       | 1           | 50   | 1.0       | 1           | NPofB |           |             |
| WT3    | 50    | 1.1       | 1           | 50   | 1.1       | 1           | NPofB |           |             |
| WT4    | 50    | 1.0       | 1           | 50   | 1.0       | 1           | NPofB |           |             |
| CC1    | 50    | 28.8      | 22          | 50   | 9.1       | 6           | 10    | 9.1       | 8.0         |
| CC2    | 50    | 1.0       | 1           | 50   | 6.7       | 4           | 10    | 10.8      | 9.0         |
| CC3    | 50    | 18.1      | 16          | 50   | 18.9      | 15          | 10    | 12.8      | 13.4        |
| CC4    | 50    | 1.0       | 1           | 0    | –         | –           | 0     | –         | –           |
| CC5    | 48    | 66.9      | 44          | 50   | 29.1      | 12          | 10    | 21.1      | 11.5        |
| CCx    | NPofB |           |             | 20   | 229.4     | 12          | 3     | 97        | 103.0       |
| SC     | 50    | 1         | 1           | 0    | –         | –           | 0     | –         | –           |

We reflect on the results and our experience in replicating the results generated by ARISTEO [1], an approximation-refinement testing technique based on surrogate models, in three consecutive editions (2020 [8], 2021 [9], and 2022 [10]) of the ARCH competition [11]. We report on our attempt to replicate the results generated from ARISTEO from editions 2020 [8], 2021 [9], and 2022 [10]. For our replication studies, we did not compare the results from the ARCH competitions with the results from [1], considering that the benchmark models are different. Menghi et al. [1] used five publicly available models

(i.e., RHB(1), RHB(2), AT, AFC, IGC) initially found by consulting the corresponding publications. The competition only uses a subset of the models from [1] (AT and AFC), and consider many additional models NN, WT, CC, F16, and SC.

#### 4.2.1. Experiment setup

Our benchmark consists of seven models: Automatic Transmission (AT) [40], Fuel Control on Automotive Powertrain (AFC) [41], Neural Network Controller (NN) [42], Wind Turbine (WT) [43], Chasing Cars (CC) [44], Aircraft Ground Collision Avoidance System (F16) [45], and Steam Condenser (SC) [46]. These Simulink models have different sizes and complexity: the number of Simulink blocks spans from 13 (CC) to 302 (AFC). The models are from different domains: automotive (AT, AFC), neural networks (NN), and energy (SC). AFC is from Toyota. The models come with different licenses, including GPLv2.

The models of the ARCH competition are associated with 27 requirements expressed using a logic-based language. Each requirement is associated with an identifier that starts with the model's name. The rows of Tables 4 and 5 represent different requirements. For example, the row with the identifier AT6a refers to the requirement AT6a of the AT model. The interested reader can find additional information about the models and requirements in the report of the ARCH competition (i.e., [8–10]).

The same configuration is considered for ARISTEO across the three years it participated in the competition. Specifically, ARISTEO is configured as follows. We used an ARX model (ARX-2) with order  $na = 2$ ,  $nb = 2$ , and  $nk = 2^1$  as the structure for the surrogate model used in the approximation-refinement loop of ARISTEO. For models with multiple inputs and outputs, the dimension of the matrix  $na$ ,  $nb$ , and  $nk$  are changed depending on the number of inputs and outputs. For the other parameters, we assigned the default configuration values of S-TaLiRo. We considered the same parametrization of S-TaLiRo for the input signals. ARISTEO executes the original Simulink model to learn the initial surrogate model. The competition sets the cut-off values for the number of simulations of the original model to 300 for 2020 and 2021, and do not impose a maximal number of simulations that can be run in one falsification trial for 2022. This choice was performed to enable a more accurate assessment of the difficulty of the benchmarks. We considered this value also for the number of simulations of the surrogate model (per trial). However, we considered 300 as the maximum number of iterations for ARISTEO in 2022 since we aimed to maintain consistency with the previous years of the competition. Notice that this choice penalizes ARISTEO over the other competing tools in 2022.

During the competition, the participants must test the model for each requirement: *piece-wise continuous input signals*, and *constrained input signals*. The first option let the participant decide the shape of the input signals, and the second option precisely fixes the format of the input signal. Tables 4 and 5 respectively report the results obtained by ARISTEO for the two options. In addition, the competition required participants to execute experiments 50 times in 2020 and 2021 and 10 times in 2022 due to the non-determinism of the underlying search-based algorithms. In 2022 the number of experiments was reduced from 50 to 10 since there was no upper bound on the maximal number of simulations that can be run in one falsification trial.

#### 4.2.2. Results

Tables 4 and 5 report the results of our experiments. For each year, they report the falsification rate with respect to the number of trials (FR), mean ( $\bar{S}$ ) and median ( $\tilde{S}$ ) number of simulations (rounded down) over successful trials (“–” if FR is zero). Cells labeled with NPofB (Not Part of the Benchmark) specifies that the model was not part of the benchmark models considered by the competition that year. For the first year (2020) of the competition, the cells for the benchmarks

<sup>1</sup> <https://nl.mathworks.com/help/ident/ref/arx.html>.

AT51, AT52, AT53, and AT54 are labeled with nSbyA (not supported by ARISTEO) since ARISTEO could not process the benchmark. Indeed, for the first year of the competition, ARISTEO could not support property specifications that use locations. Locations are used by S-TaLiRo [35] (the tool ARISTEO extends) to specify properties that involve variables that can assume values within a finite set of values.

The results from Table 4 show that the falsification rates for AT1, AT2, AT51, AT52, AT53, AT6abc, WT1, WT2, WT3, WT4, CC1, CC2, CC3, CC5, F16 are the same for all the three years of the competition. The mean ( $\bar{S}$ ) and median ( $\tilde{S}$ ) number of simulations (rounded down) over successful trials are also consistent for these benchmarks (small fluctuations are caused by the non-determinism of the underlying search-based algorithm). For AT54, the property was not falsified in 2022 while it was falsified in 3 cases in 2021. However, unlike 2021 that require the experiments to be executed 50 times, in 2022 experiments were only executed 10 times. For AT6a, AT6b, AT6c, and CCX small fluctuations in the falsification rate, mean ( $\bar{S}$ ) and median ( $\tilde{S}$ ) number of simulations are caused by the non-determinism of the underlying search-based algorithm which may produce different results across multiple experiments. For NN, we noticed that the results of 2022 are significantly different from the ones obtained in 2020 and 2021. Unlike 2022, in 2020 and 2021, there was a mistake in the specification of the requirement: there was a typo in one of the temporal operators used to specify the requirement ( $\square_{[1,18]}$  instead of  $\square_{[1,37]}$ ). For CC4 and SC, the results of 2020 (the first year in which ARISTEO participated in the competition) are significantly different from the ones obtained in 2021 and 2022. For CC4, in 2020, the predicate was written incorrectly:  $y_4 - y_5 \geq 8$  instead of  $y_5 - y_4 \geq 8$ . The former is immediately falsified. Furthermore, the simulation time in 2020 was 30 s, which was lower than the temporal operators of the requirement (in 2021–22 it was 100 s). For SC, in 2020, ARISTEO used 12 control points and a simulation time of 100 s. In 2021–22, ARISTEO used 20 control points, and the simulation time was reduced to 35 s. Furthermore, for piecewise continuous input signals ARISTEO reports the results for constrained input signals: the piecewise continuous input signals let the participant be free to decide the shape of their input signals, and it was decided to use the one mandated by the constrained input signals.

The results from Table 5 show that the falsification rates for AT1, AT2, AT51, AT53, AT54, AFC29, NN, WT1, WT2, WT3, WT4, CC1, CC2, CC3, are the same for all the three years of the competition. The mean ( $\bar{S}$ ) and median ( $\tilde{S}$ ) number of simulations (rounded down) over successful trials are also consistent for these benchmarks (small fluctuations are caused by the non-determinism of the underlying search-based algorithm). For AT52, AT6a, AT6b, AT6c, AT6abc, AFC27, and CC5 small fluctuations in the falsification rate, mean ( $\bar{S}$ ) and median ( $\tilde{S}$ ) number of simulations are caused by the non-determinism of the underlying search-based algorithm which may produce different results across multiple runs. For AFC33, we noticed that the results of 2022 are significantly different from the ones obtained in 2020 and 2021 since a wrong input range was considered in 2020 and 2021. In 2020 and 2021, the input range of AFC27 and AFC29 was erroneously considered. In 2020, the wrong input range was considered (there was a bug in the code that was supposed to change the range). In 2021, the input range was correct, but the control points were considered in the wrong order. For NNx, the results of 2022 are significantly different from the ones obtained in 2021 (NNx was not part of the benchmark models in 2020). NNx has a different input range compared to NN. However, in 2021, the input range for NN was erroneously considered. For CC4, the results of 2020 are significantly different from the ones obtained in 2021 and 2022. As previously reported, the cause was a problem in the specification of one of the predicates and a different simulation time. For SC, the success rate for 2022 should be 0; there was a typo on the report for ARCH 2022. In addition, for SC, in 2020, ARISTEO considered a different simulation time and a different number of control points.

## 5. Lessons learned

In this section, we reflect on the lessons learned from our replication studies and the creation of a taxonomy for the SA-SBT literature. Our lessons concern the methods and metrics used to evaluate SA-SBT solutions, the scope and purpose of using surrogates in SBT, and the need for frameworks to facilitate extensive experimentation in this domain. We believe our lessons would be most relevant for software engineering researchers and practitioners working on the verification and testing of cyber-physical and autonomous systems.

### 5.1. Metrics for evaluating SA-SBT

The evaluation metrics used in the subject papers and the research reviewed in Section 3 can be grouped into three categories: (1) *Metrics to assess the effectiveness of SA-SBT*. The effectiveness of a SA-SBT solution is evaluated by its ability to identify failures in the SUT and is quantified by the number of distinct failure scenarios it detects and the severity of each scenario, as indicated by the fitness function values. (2) *Metrics to assess the efficiency of SA-SBT*. As discussed in Section 2 under the *evaluation metrics* category, the efficiency of SA-SBT techniques is typically evaluated based on the number of iterations, execution time and estimated execution time. (3) *Metrics to assess the performance SA-SBT solutions that rely on Pareto-based search algorithms*. These include metrics such as HV, Spread and GD discussed in Section 4.1. We provide three lessons discussed below corresponding to these three metric categories. These lessons aim to identify the limitations of the current metrics when applied to SA-SBT solutions, and suggest alternatives for improvement.

*Lesson 1: The effectiveness of a testing algorithm should be evaluated based on its ability in identifying distinct and meaningful failures.* Papers on software testing often report the number of generated failures as a metric to evaluate effectiveness (e.g., [6,23]). However, for SA-SBT solutions, particularly when they are applied to case studies such as ADAS, it is important to report failures that are *distinct* and *valid*. Failures are considered distinct when they are generated by test inputs that are significantly different. In the study that we replicated in Section 4.1, distinct scenarios are defined as those that differ in at least one input value [6]. While this definition might be sufficient for the AEB and AVP case studies in Section 4.1, it may fall short in other situations where two test inputs may differ in variables that are not related to failures. For example, we may have two pedestrians in the initial scene where one is far away from the collision. Changing the position of the far-away pedestrian would not help reveal interesting and diverse failure situations. In addition, small variations in continuous input variables may not reveal diverse failures. For example, in Fig. 3, slightly moving the pedestrian to the left or right in a failing test input may still lead to failures that are essentially the same. In a recent study, Zhong et al. [22] proposed an improved definition of distinctness that requires variations in a minimum number of input variables and differences between continuous variables to exceed a user-defined threshold. However, this definition relies on user-defined parameters that can vary between domains, making it difficult to replicate and generalize the evaluation results.

In addition to being distinct, failures should be valid and meaningful. For example, both AVP and AEB fail if the car speed is higher than the speed limit for urban areas (i.e., 100 km/h). To ensure the validity of failures, it is important that we select the input variable ranges such that they satisfy the expected preconditions of the SUT. Unmet preconditions may cause failures that are not faults in the SUT, but rather caused by invalid inputs. The validity of failures, further, depends on the way we formalize the notion of failure or the way we define test oracles. In our NSGAII-DT study, the criticality function acts as a test oracle and determines whether, or not, a scenario is a failure. For the AEB case study, the criticality function was defined using a conjunction of predicates over fitness functions that are capped

by some thresholds. The formalization of the fitness functions, however, had limitations which led to some invalid failures, e.g., the pedestrian hitting the car from the side [5,6]. We addressed this issue, in our replication study (Section 4.1), by redefining the fitness functions such that scenarios where the pedestrians approach the ego car from the sides are penalized and are not considered as critical.

**Lesson 2: The accuracy of statistical tests can be impacted by the low number of repetitions.** A primary research question for evaluating SA-SBT techniques is to assess whether they *improve the efficiency of testing while ensuring its effectiveness*. All the SA-SBT techniques we reviewed in Table 1 answer this research question by comparing SA-SBT with some non-surrogate SBT baseline. They use different metrics for this comparison though. Specifically, all the approaches, except for Menghi et al. [1], use the *execution time* metric. That is, they execute the SA-SBT and the baseline techniques for the same amount of time and compare the results. In all these cases, however, the compute-intensive study subjects used for evaluation are simulators or Simulink models that take between two to ten minutes to execute. When the execution times of the study subjects are in the order of a few minutes, it is still feasible to execute the SA-SBT and the baseline several times and draw conclusions using statistical tests. It is, however, important to note that, in these papers, the number of times experiments were repeated to account for randomness was rather low, i.e., between 10 to 15 times. This can negatively impact the accuracy of the statistical test results. Further, none of these papers performed any extensive meta-evaluations or hyper-parameter optimizations.

In the case of Menghi et al. [1], a single simulation of the compute-intensive subject takes 1.5 h, and the approach of the paper which involves the use of SI requires extensive hyper-parameter optimizations. Performing all the required experiments on their compute-intensive subject could take in the order of 50 years to complete. Hence, Menghi et al. [1] perform a part of their experiments on non-compute-intensive subjects, measure the number of iterations required to detect failures and compute the *estimated execution time*, i.e., the estimated time that performing these numbers of iterations would require for compute-intensive models. The testing techniques are then compared by considering the *estimated execution time* instead of the same *execution time*.

**Lesson 3: Among the Pareto-based quality indicators considered in our prior research, HV is the most appropriate one for assessing SA-SBT algorithms.** The quality indicators typically determine how fast the outputs of a Pareto-based algorithm converge towards an ideal or true Pareto front. These indicators may not directly relate to the main objective of a test generation algorithm though. Hence, there needs to be a justification as to why a quality indicator is used to assess a test generation algorithm. In our original study we selected three indicators, HV, Spread and GD, based on their widespread use and their collective ability to evaluate complementary aspects of convergence, spread, and uniformity [47]. As a result of our replication, we have reached at two observations related to the use of quality indicators for assessing Pareto-based test generation algorithms: (1) GD and Spread metrics are most meaningful for comparing population sets with equal sizes. Spread, while being normalized to the size of a population set, provides different values for two sets of different sizes, in which individuals are equally separated, which might lead to wrong interpretation of the spread for comparing populations of the same algorithm generated over subsequent runs. Even though GD does not have exactly the same problem, introduction of another individual to a population set of a small size might heavily worsen the metric value, and therefore, might also lead to incorrect interpretation of convergence over a run. We note that in contrast to classical Pareto optimization algorithms (e.g. NSGAII), NSGAII-DT allows for a flexible population size during the search. Relaxing the population size in search algorithms employed for testing is common not only for SA-SBT techniques but also in the many-objective search algorithms [48,49]. The flexibility in the population size allows for more targeted exploration guided by surrogates,

or to maintain a subset of the solutions on an optimal Pareto that are useful for the testing problem at hand. However, flexible population sizes can also affect metrics such as Spread and GD leading to potential misinterpretation of the results. (2) Among the three quality indicators used in our original study, the results related to HV in the replication study are the most faithful to those obtained from the original study. This might be partly due to the issue that Spread and GD are not suitable metrics for comparing Pareto-based algorithms with varying populations while HV is not dependent on the population size. Overall, in our experience, HV showed as a better metric for assessing our proposed Pareto-based SA-SBT test generation algorithms.

## 5.2. Surrogates for fault localization and repair

The main objective of existing SA-SBT approaches is to generate test cases that can effectively reveal failures. Empirical evidence from our subject papers [1,4–6], the research reviewed in Section 3 and our two replication studies support the idea that the incorporation of surrogates into SBT significantly enhances the effectiveness and efficiency of automated testing in revealing failures. However, it is important to ensure that the techniques do not generate spurious failures, e.g., failures caused by invalid inputs and not due to system faults or failures that represent unrealistic situations. In our paper introducing NSGAII-DT, we utilized surrogate models to identify input conditions that led to failures, which we then validated against domain knowledge [6]. These conditions are valuable because they can help explain why failures occurred, which in turn can aid in root-cause analysis, fault localization, and bug fixing. However, we did not leverage these conditions to distinguish between valid and spurious failures, nor did we use them to identify root causes of failures. To the best of our knowledge, no prior research has explored the potential of surrogate models in explaining failure conditions. We believe that these models can serve as effective tools for identifying spurious failures, as well as developing fault localization and repair strategies.

## 5.3. Frameworks for large-scale experimentation

Further advancements in the area of SA-SBT require systematic evaluation frameworks that promote accuracy, transparency and reproducibility. Our first lessons-learned highlighted the challenge of defining proper evaluation metrics, but there is also the additional obstacle of limited platforms and benchmarks available for large-scale experimentation with SA-SBT methods. As discussed in our taxonomy (Section 2), SA-SBT methods may be applied in the MiL, SiL and HiL contexts, but the main focus of the research so far has been on MiL testing. The majority of MiL testing has been performed on case studies based on simulation-based testing of autonomous driving systems or Simulink models. A variety of simulators and ADAS components have been used in the literature for simulation-based testing of autonomous driving systems. The example simulators include: PreScan [29], ProSiVIC [50], CARLA [51], and BeamNG [52], and the ADAS components include both industrial and proprietary ADAS and open source DNN-enabled components. A framework is needed to enable experimentation and evaluation of testing algorithms using different simulators and ADAS alternatives, as most current test generation algorithms are evaluated on a small subset, or even a single, simulator and ADAS. Our replication in Section 4.1 concerned reproducing results from an original study using the same simulator but two different ADAS. As shown there, a number of different factors need to be considered when we apply a heuristic algorithm to two different ADAS even when the simulator is the same. Other replication attempts in the context of ADAS testing include the work of Borg et al. [53] on replicating the results of a search-based testing algorithm on two different simulators (i.e., ProSiVic and PreScan) for the same ADAS component, and the work of Stocco et al. [54] on generalizing the testing results obtained using an ADAS simulator to a physical platform. Both studies observe



notable discrepancies between the testing results obtained from the different simulators and platforms. In particular, the latter work which is a rare example of transferring the testing result from MiL to HiL shows some major reality gaps between the virtual and physical world.

#### 5.4. International competitions

International competitions are a successful instrument to support experiment replication effectively. While participating in competitions requires significant effort, these competitions offer several benefits. We learned many lessons over our three-year participation in the ARCH competition that we summarize below.

*Forces tools updates.* Authors need to constantly update their tools to ensure specific formatting of their inputs and outputs to adhere to the rules of the competition. From our experience, we learned that this activity improved our tool. For example, participating in the competition in 2021 enabled us to extend the implementation of ARISTEO to support property specifications that use locations. This activity extended the support of ARISTEO to benchmarks AT51, AT52, AT53, and AT54.

*Supports experiment replication and verification of previous results.* Replicating previous experiments is not always a trivial and straightforward activity. Replicating the experiments for AT51, AT52, AT54, and AT54 was not trivial. ARISTEO is a plugin for S-TaLiRo. S-TaLiRo requires to specify these requirements using “control locations” ( $\text{Pred}(i).\text{loc}$ ), i.e., specific constructs used to establish fitness metrics of hybrid systems. Despite S-TaLiRo being widely documented and the explanations being detailed and exhaustive, the documentation of this construct should be improved. Specifically, to analyze these requirements, we had to install MatlabBGL [55], a library containing a set of algorithms to work with graphs, and recompile the C++ files that are used to compute the fitness metric using an old version (1\_36\_0 [56]) of Boost, a portable C++ and GCC library. However, this needed to be more precisely detailed since we had to reverse engineer the version of the library to be used: we could not use newer versions of Boost since they are not retro-compatible for this case. ARISTEO participation was managed by different students across the years. We spent significant time in 2021 adding support for these requirements by extending the implementation of ARISTEO and understanding how to install the different libraries. However, we did not precisely document our activity, and we had to perform the same reverse engineering process in 2022. We now have a detailed description of the libraries that need to be installed to support these requirements, add it to our documentation, and contact the developers of S-TaLiRo to extend their documentation. Finally, as detailed in Section 4.2, participating in consecutive years of the competition enables spotting configuration errors and improving the reliability of the published results.

*Discovering new results over time.* The set of benchmark models change over time: the ARCH competition is organized as part of the International Workshop on Applied Verification of Continuous and Hybrid Systems, which contains an explicit call for new benchmark models. For example, in 2022, the organizers removed the WT model from the set of benchmark models considered in the competition, and the pacemaker model [57] was proposed at the workshop and is likely to be added to the set of benchmark models to be considered in future editions of the competition. The variation of the benchmark models enables a continuous comparison and assessment of the tools on models from different domains and with various characteristics (e.g., number of blocks).

*Supports networking and research collaborations.* The ARCH competition is a friendly competition with the primary purpose of keeping researchers and practitioners updated with the latest advancements in the field. It is designed to support and foster networking and research collaborations. This decision ensures that participants are offered a friendly and cordial environment. We learned that this offered a great opportunity (especially for students) for networking, getting new research ideas, and establishing research collaborations.

Finally, we learned that organizing and managing the competition is a significant undertaking. The coordinator of the Falsification Category of the ARCH Competition (Gidon Ernst) did a significant job across the three years in which we participated to ensure the competition's success. Organizing the meetings to define the rules of the competition, collecting and analyzing the results, leading the writing of the competition report, and reporting the results at the workshop are a significant service to the community that should be acknowledged and appropriately rewarded.

Our taxonomy can support the organizers of the ARCH competition by supporting activities such as adding new benchmarks. For example, the organizers may extend the benchmarks to include models with more complexity and longer execution time. Further, the metrics we discussed earlier in this section can be used as part of the benchmark and competition since, currently, the number of iterations is the only metric used for the tool competition.

## 6. Conclusions

This paper reflects on four papers published between 2014 and 2021 [1,4–6] that propose surrogate-assisted search-based testing (SA-SBT) techniques for autonomous systems. We developed a taxonomy based on our synthesis of different SA-SBT approaches. Our taxonomy identifies the main dimensions of SA-SBT solutions and we demonstrate how different SA-SBT solutions can be positioned along these dimensions. We report on two replication studies using an industrial advanced driver assistance system (ADAS) and a benchmark of Simulink models. The results of our taxonomy and replication studies highlight the need for improvement in the metrics used to evaluate SA-SBT, the potential for using surrogates in fault localization and repair, and the importance of benchmarking, international competitions and creating frameworks for large-scale experiments involving SA-SBT techniques.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Data availability

Data will be made available on request.

### Acknowledgments

We acknowledge Gidon Ernst for leading the Falsification Category of the ARCH Competition in 2020, 2021, and 2022. We also acknowledge Khoulood Gaaloul that supported the participation of ARISTEO in the 2020 and 2021 editions of the ARCH competition.

We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC) [funding reference numbers DGEER-2020-00261, RGPIN-2022-04622, DGEER-2022-0040].

 This research paper has further received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 956123.

## References

- [1] C. Menghi, S. Nejati, L. Briand, Y.I. Parache, Approximation-refinement testing of compute-intensive cyber-physical models: An approach based on system identification, in: *IEEE/ACM 42nd International Conference on Software Engineering, ICSE, 2020*, pp. 372–384.
- [2] A. Zeller, Search-based testing and system testing: A marriage in heaven, in: *International Workshop on Search-Based Software Testing, (SBST@ICSE), IEEE/ACM, 2017*, pp. 49–50.
- [3] Y. Jin, Surrogate-assisted evolutionary computation: Recent advances and future challenges, *Swarm Evol. Comput.* 1 (2) (2011) 61–70.



- [4] R. Matinnejad, S. Nejati, L.C. Briand, T. Bruckmann, Mil testing of highly configurable continuous controllers: scalable search using surrogate models, in: I. Crnkovic, M. Chechik, P. Grünbacher (Eds.), *International Conference on Automated Software Engineering (ASE)*, ACM/IEEE, 2014, pp. 163–174.
- [5] R.B. Abdesslem, S. Nejati, L.C. Briand, T. Stifter, Testing advanced driver assistance systems using multi-objective search and neural networks, in: *International Conference on Automated Software Engineering (ASE)*, IEEE/ACM, 2016, pp. 63–74.
- [6] R.B. Abdesslem, S. Nejati, L.C. Briand, T. Stifter, Testing vision-based control systems using learnable evolutionary algorithms, in: M. Chaudron, I. Crnkovic, M. Chechik, M. Harman (Eds.), *International Conference on Software Engineering, (ICSE)*, ACM, 2018, pp. 1016–1026.
- [7] E.M. Clarke, A. Biere, R. Raimi, Y. Zhu, Bounded model checking using satisfiability solving, *Form. Methods Syst. Des.* 19 (1) (2001) 7–34.
- [8] G. Ernst, P. Arcaini, I. Bennani, A. Donzé, G. Fainekos, G. Frehse, L. Mathesen, C. Menghi, G. Pedrielli, M. Pouzet, S. Yaghoubi, Y. Yamagata, Z. Zhang, ARCH-COMP 2020 category report: Falsification, in: *International Workshop on Applied Verification of Continuous and Hybrid Systems*, 74, EasyChair, 2020, pp. 140–152.
- [9] I.B.A.C. G. Ernst, A. Donzé, G. Fainekos, G. Frehse, K. Gaaloul, J. Inoue, T. Khandait, L. Mathesen, C. Menghi, G. Pedrielli, M. Pouzet, M. Waga, S. Yaghoubi, Y. Yamagata, Z. Zhang, ARCH-COMP category report: Falsification with validation of results, in: *Workshop on Applied Verification of Continuous and Hybrid Systems*, Vol. 80, EasyChair, 2021, pp. 133–152.
- [10] G. Ernst, P. Arcaini, G. Fainekos, F. Formica, J. Inoue, T. Khandait, M.M. Mahboob, C. Menghi, G. Pedrielli, M. Waga, Y. Yamagata, Z. Zhang, ARCH-comp 2022 category report: Falsification with unbounded resources, in: *International Workshop on Applied Verification of Continuous and Hybrid Systems (ARCH22)*, in: *EPiC Series in Computing*, 90, EasyChair, 2022, pp. 204–221, <http://dx.doi.org/10.29007/fhmk>.
- [11] International competition on verifying continuous and hybrid systems, 2022, [Online] URL <https://cps-vo.org/group/ARCH/FriendlyCompetition>.
- [12] T. Dybå, N.A.M. Maiden, R. Glass, The reflective software engineer: Reflective practice, *IEEE Softw.* 31 (4) (2014) 32–36.
- [13] S. Luke, *Essentials of Metaheuristics*, second ed., Lulu, 2013, Available for free at <http://cs.gmu.edu/~sean/book/metaheuristics/>.
- [14] H. Beglerovic, M. Stolz, M. Horn, Testing of autonomous vehicles using surrogate models and stochastic optimization, in: *International Conference on Intelligent Transportation Systems (ITSC)*, IEEE, 2017, pp. 1–6.
- [15] A. Arrieta, S. Wang, U. Markiegi, G. Sagardui, L. Etxeberria, Search-based test case generation for cyber-physical systems, in: *Congress on Evolutionary Computation (CEC)*, IEEE, 2017, pp. 688–697.
- [16] Y. Wang, R. Yu, S. Qiu, J. Sun, H. Farah, Safety performance boundary identification of highly automated vehicles: A surrogate model-based gradient descent searching approach, *IEEE Trans. Intell. Transp. Syst.* (2022) 1–12, <http://dx.doi.org/10.1109/TITS.2022.3191088>.
- [17] C. Innes, S. Ramamoorthy, Automated testing with temporal logic specifications for robotic controllers using adaptive experiment design, in: *International Conference on Robotics and Automation (ICRA)*, IEEE, 2022, pp. 6814–6821, <http://dx.doi.org/10.1109/ICRA46639.2022.9811579>.
- [18] Z. Zhang, P. Arcaini, Gaussian process-based confidence estimation for hybrid system falsification, in: M. Huisman, C.S. Pasareanu, N. Zhan (Eds.), *Formal Methods (FM)*, Springer, 2021, pp. 330–348.
- [19] G. Pedrielli, T. Khandait, S. Chotailiya, Q. Thibault, H. Huang, M. Castillo-Effen, G. Fainekos, Part-x: A family of stochastic algorithms for search-based test generation with probabilistic guarantees, 2021, *CoRR abs/2110.10729*. URL: <http://arxiv.org/abs/2110.10729>, [arXiv:2110.10729](https://arxiv.org/abs/2110.10729).
- [20] D. Humeniuk, G. Antoniol, F. Khomh, Data driven testing of cyber physical systems, 2021, *CoRR abs/2102.11491*. URL: <http://arxiv.org/abs/2102.11491>, [arXiv:2102.11491](https://arxiv.org/abs/2102.11491).
- [21] D. Humeniuk, F. Khomh, G. Antoniol, A search-based framework for automatic generation of testing environments for cyber-physical systems, *Inf. Softw. Technol.* 149 (2022) 106936, <http://dx.doi.org/10.1016/j.infsof.2022.106936>.
- [22] Z. Zhong, G.E. Kaiser, B. Ray, Neural network guided evolutionary fuzzing for finding traffic violations of autonomous vehicles, 2021, *CoRR abs/2109.06126*. URL: <http://arxiv.org/abs/2109.06126>, [arXiv:2109.06126](https://arxiv.org/abs/2109.06126).
- [23] F.U. Haq, D. Shin, L. Briand, Efficient online testing for DNN-enabled systems using surrogate-assisted and many-objective optimization, in: *International Conference on Software Engineering (ICSE 2022)*, 2022, pp. 811–822.
- [24] H.L. da Silva Araujo, T. Hoenselaar, M.R. Mousavi, A.V. Vinel, Connected automated driving: A model-based approach to the analysis of basic awareness services, in: *31st IEEE Annual International Symposium on Personal, Indoor and Mobile Radio Communications, PIMRC 2020*, London, United Kingdom, August 31 - September 3, 2020, IEEE, 2020, pp. 1–7, <http://dx.doi.org/10.1109/PIMRC48278.2020.9217142>.
- [25] H.L. da Silva Araujo, G. Carvalho, M.R. Mousavi, A. Sampaio, Multi-objective search for effective testing of cyber-physical systems, in: P.C. Ölveczky, G. Salaün (Eds.), *Software Engineering and Formal Methods - 17th International Conference, SEFM 2019*, Oslo, Norway, September 18–20, 2019, Proceedings, in: *Lecture Notes in Computer Science*, 11724, Springer, 2019, pp. 183–202, [http://dx.doi.org/10.1007/978-3-030-30446-1\\_10](http://dx.doi.org/10.1007/978-3-030-30446-1_10).
- [26] S. Bittanti, *Model Identification and Data Analysis*, John Wiley & Sons, 2019.
- [27] M.M. AMS, P. Electromagnetism, *System identification*, 1989.
- [28] FOCETA, 2023. URL: <http://www.foceta-project.eu/> accessed: 2023-02-02.
- [29] PreScan, Simcenter prescan, 2023, URL <https://www.plm.automation.siemens.com/global/de/products/simcenter/prescan.html>. Accessed: 2023-02-02.
- [30] M.D. McKay, R.J. Beckman, W.J. Conover, A comparison of three methods for selecting values of input variables in the analysis of output from a computer code, *Technometrics* 21 (2) (1979) 239–245, URL <http://www.jstor.org/stable/1268522>.
- [31] C. Henard, M. Papadakis, G. Perrouin, J. Klein, Y.L. Traon, PLEDGE: a product line editor and test generation tool, in: *International Software Product Line Conference Co-located Workshops (SPLC'13)*, ACM, 2013, pp. 126–129.
- [32] T. Chen, M. Li, X. Yao, How to evaluate solutions in Pareto-based search-based software engineering? A critical review and methodological guidance, 2020, *CoRR abs/2002.09040*. URL: <http://arxiv.org/abs/2002.09040>, [arXiv:2002.09040](https://arxiv.org/abs/2002.09040).
- [33] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Trans. Evol. Comput.* 6 (2) (2002) 182–197, <http://dx.doi.org/10.1109/4235.996017>.
- [34] NSGAI-DT Replication, Results replication NSGAI-DT, 2023, URL [https://github.com/Leviathan321/reflection\\_study](https://github.com/Leviathan321/reflection_study).
- [35] Y. Annpureddy, C. Liu, G. Fainekos, S. Sankaranarayanan, S-TaLiRo: A tool for temporal logic falsification for hybrid systems, in: *Tools and Algorithms for the Construction and Analysis of Systems*, Springer, 2011, pp. 254–257.
- [36] M. Waga, Falsification of cyber-physical systems with robustness-guided black-box checking, in: *International Conference on Hybrid Systems: Computation and Control*, ACM, 2020, pp. 11:1–11:13.
- [37] Y. Yamagata, S. Liu, T. Akazaki, Y. Duan, J. Hao, Falsification of cyber-physical systems using deep reinforcement learning, *IEEE Trans. Softw. Eng.* 47 (12) (2021) 2823–2840, <http://dx.doi.org/10.1109/TSE.2020.2969178>.
- [38] G. Ernst, S. Sedwards, Z. Zhang, I. Hasuo, Fast falsification of hybrid systems using probabilistically adaptive input, 2018, *arXiv preprint arXiv:1812.04159*.
- [39] Z. Zhang, D. Lyu, P. Arcaini, L. Ma, I. Hasuo, J. Zhao, Effective hybrid system falsification using Monte Carlo tree search guided by QB-robustness, in: A. Silva, K.R.M. Leino (Eds.), *Computer Aided Verification*, Springer, 2021, pp. 595–618.
- [40] B. Hoxha, H. Abbas, G. Fainekos, Benchmarks for temporal logic requirements for automotive systems, in: G. Frehse, M. Althoff (Eds.), *ARCH14-15. 1st and 2nd International Workshop on Applied Verification for Continuous and Hybrid Systems*, in: *EPiC Series in Computing*, 34, EasyChair, 2015, pp. 25–30, <http://dx.doi.org/10.29007/xwrs>.
- [41] X. Jin, J.V. Deshmukh, J. Kapinski, K. Ueda, K. Butts, Powertrain control verification benchmark, in: *International Conference on Hybrid Systems: Computation and Control, HSCC '14*, ACM, 2014, pp. 253–262, <http://dx.doi.org/10.1145/2562059.2562140>.
- [42] NARMA-L2, Design NARMA-L2 Neural Controller in Simulink, 2022, <https://www.mathworks.com/help/deeplearning/ug/design-narma-l2-neural-controller-in-simulink.html>. Accessed: 2022-11-21.
- [43] S. Schuler, F.D. Adegas, A. Anta, Hybrid modelling of a wind turbine, in: G. Frehse, M. Althoff (Eds.), *ARCH16. International Workshop on Applied Verification for Continuous and Hybrid Systems*, in: *EPiC Series in Computing*, 43, EasyChair, 2017, pp. 18–26, <http://dx.doi.org/10.29007/tf1p>.
- [44] J. Hu, J. Lygeros, S. Sastry, Towards a theory of stochastic hybrid systems, in: *International Workshop on Hybrid Systems: Computation and Control*, Springer, 2000, pp. 160–173.
- [45] P. Heidlauf, A. Collins, M. Bolender, S. Bak, Verification challenges in F-16 ground collision avoidance and other automated maneuvers, in: G. Frehse (Ed.), *ARCH18. 5th International Workshop on Applied Verification of Continuous and Hybrid Systems*, in: *EPiC Series in Computing*, 54, EasyChair, 2018, pp. 208–217, <http://dx.doi.org/10.29007/91x9>.
- [46] S. Yaghoubi, G. Fainekos, Gray-box adversarial testing for control systems with machine learning components, in: *International Conference on Hybrid Systems: Computation and Control, HSCC*, 2019.
- [47] J. Wu, P. Arcaini, T. Yue, S. Ali, H. Zhang, On the preferences of quality indicators for multi-objective search algorithms in search-based software engineering, *Empir. Softw. Eng.* 27 (6) (2022) 144, <http://dx.doi.org/10.1007/s10664-022-10127-4>.
- [48] R.B. Abdesslem, A. Panichella, S. Nejati, L.C. Briand, T. Stifter, Testing autonomous cars for feature interaction failures using many-objective search, in: M. Huchard, C. Kästner, G. Fraser (Eds.), *International Conference on Automated Software Engineering (ASE)*, ACM/IEEE, 2018, pp. 143–154.
- [49] A. Panichella, F.M. Kifetew, P. Tonella, Reformulating branch coverage as a many-objective optimization problem, in: *International Conference on Software Testing, Verification and Validation, ICST*, IEEE Computer Society, 2015, pp. 1–10.
- [50] A. Belbachir, J.-C. Smal, J.-M. Blosseville, D. Gruyer, Simulation-Driven Validation of Advanced Driving-Assistance Systems, *Transport Research Arena 2012, Procedia - Social and Behavioral Sciences* 48 (2012) 1205–1214, <http://dx.doi.org/10.1016/j.sbspro.2012.06.1096>.
- [51] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator, 2017, *arXiv preprint arXiv:1711.03938*.

- [52] A. Gambi, M. Müller, G. Fraser, AsFault: testing self-driving car software using search-based procedural content generation, in: J.M. Atlee, T. Bultan, J. Whittle (Eds.), *International Conference on Software Engineering: Companion Proceedings, ICSE, IEEE / ACM*, 2019, pp. 27–30.
- [53] M. Borg, R.B. Abdessalem, S. Nejati, F. Jegeden, D. Shin, Digital twins are not monozygotic - cross-replicating ADAS testing in two industry-grade automotive simulators, in: *Conference on Software Testing, Verification and Validation (ICST)*, IEEE, 2021, pp. 383–393.
- [54] A. Stocco, B. Pulfer, P. Tonella, Mind the gap! a study on the transferability of virtual vs physical-world testing of autonomous driving systems, *IEEE Trans. Softw. Eng.* (2022) 1–13, <http://dx.doi.org/10.1109/tse.2022.3202311>.
- [55] MatlabBGL, MatlabBGL, 2022, <https://www.mathworks.com/matlabcentral/fileexchange/10922-matlabagl>. Accessed: 2022-11-21.
- [56] BOOST, BOOST, 2022, <https://sourceforge.net/projects/boost/files/boost/1.36.0/>. Accessed: 2022-11-21.
- [57] M. Ayesh, N. Mehan, E. Dhanraj, A. El-Rahwan, S.E. Opalka, T. Fan, A. Hamilton, A.M. Jacob, R.A. Sundarajan, B. Widjaja, C. Menghi, Two simulink models with requirements for a simple controller of a pacemaker device, in: *International Workshop on Applied Verification of Continuous and Hybrid Systems (ARCH22)*, in: *EPiC Series in Computing*, 90, EasyChair, 2022, pp. 18–25.